ASSIGNMENT-12.5

HT.NO-2403A52L06

BATCH-50

Q: Lab 12: Algorithms with AI Assistance – Sorting, Searching, and Optimizing Algorithms Lab Objectives:

• Apply AI-assisted programming to implement and optimize Week6 - Friday sorting and searching algorithms.

• Compare different algorithms in terms of efficiency and use cases.

• Understand how AI tools can suggest optimized code and complexity improvements.

Task Description #1 (Sorting – Merge Sort Implementation) • Task:

Use AI to generate a Python program that implements the

Merge Sort algorithm.

• Instructions: o Prompt AI to create a function merge_sort(arr) that sorts a list in ascending order.

o Ask AI to include time complexity and space

complexity in the function docstring. o Verify the

generated code with test cases.

• Expected Output: o A functional Python script implementing Merge Sort with proper documentation.

Task Description #2 (Searching – Binary Search with AI

Optimization)

• Task: Use AI to create a binary search function that finds a target element in a sorted list.

• Instructions:

o Prompt AI to create a function binary_search(arr, target) returning the index of the target or -1 if not found.

o Include docstrings explaining best, average, and worst-case complexities. o Test with various inputs. • Expected Output:

o Python code implementing binary search with AI-generated comments and docstrings.

Task Description #3: Smart Healthcare Appointment Scheduling System

A healthcare platform maintains appointment records containing appointment ID, patient name, doctor name, appointment time, and consultation fee. The system needs to:

1. Search appointments using appointment ID.

2. Sort appointments based on time or consultation fee.

Student Task

• Use AI to recommend suitable searching and sorting algorithms.

• Justify the selected algorithms.

• Implement the algorithms in Python.

Task Description #4: Railway Ticket Reservation System

Scenario

A railway reservation system stores booking details such as ticket ID, passenger name, train number, seat number, and travel date. The system must:

1. Search tickets using ticket ID.

2. Sort bookings based on travel date or seat number.

Student Task

• Identify efficient algorithms using AI assistance.

• Justify the algorithm choices.

• Implement searching and sorting in Python.

Task Description #5: Smart Hostel Room Allocation System A hostel management system stores student room allocation details including student ID, room number, floor, and allocation date. The system needs to:

1. Search allocation details using student ID.
2. Sort records based on room number or allocation date.

Student Task

• Use AI to suggest optimized algorithms.
• Justify the selections.
• Implement the solution in Python.

Task Description #6: Online Movie Streaming Platform A streaming service maintains movie records with movie ID, title, genre, rating, and release year. The platform needs to:

1. Search movies by movie ID.
2. Sort movies based on rating or release year.

Student Task

• Recommend searching and sorting algorithms using AI.
• Justify the chosen algorithms.
• Implement Python functions.

Task Description #7: Smart Agriculture Crop Monitoring System

An agriculture monitoring system stores crop data with crop ID, crop name, soil moisture level, temperature, and yield estimate. Farmers need to:

1. Search crop details using crop ID.
2. Sort crops based on moisture level or yield estimate.

Student Task

• Use AI-assisted reasoning to select algorithms.
• Justify algorithm suitability.

• Implement searching and sorting in Python.

Task Description #8: Airport Flight Management System An airport system stores flight information including flight ID, airline name, departure time, arrival time, and status. The system must:

1. Search flight details using flight ID.

2. Sort flights based on departure time or arrival time.

Student Task

• Use AI to recommend algorithms.

• Justify the algorithm selection.

• Implement searching and sorting logic in Python.

CODE:

```
# ==============================================================================
# TASK 1: MERGE SORT IMPLEMENTATION
# ==============================================================================


def merge_sort(arr):
    """
    Sorts a list in ascending order using Merge Sort algorithm.

    Time Complexity: O(n log n) - Best, Average, and Worst case
    Space Complexity: O(n) - Additional space required for temporary arrays

    Args:
        arr (list): List of comparable elements to sort
```

```
    Returns:        list: Sorted list in
ascending order
    """     if len(arr)
<= 1:        return arr

    mid = len(arr) // 2     left = merge_sort(arr[:mid])
right
= merge_sort(arr[mid:])

    return merge(left, right)

def merge(left, right):
    """
    Merges two sorted lists into a single sorted list.

    Args:
        left (list): First sorted list        right
(list): Second sorted list

    Returns:        list:
Merged sorted list
    """
result = []     i
= j = 0 while i
< len(left)
```

```
and j < len(right):     if left[i]
<= right[j]:
result.append(left[i])  i += 1
else:
            result.append(right[j])  j
+= 1

    result.extend(left[i:])
result.extend(right[j:])     return
result

# Test cases for Merge Sort print("=" *
70) print("TASK 1: MERGE SORT
TESTING") print("=" * 70) test_arrays
= [
    [64, 34, 25, 12, 22, 11, 90],
    [5, 2, 8, 1, 9],
    [1],
    [],
    [10, 10, 10, 10]
]

for test_arr in test_arrays:    print(f"Original:
 {test_arr}") print(f"Sorted:

{merge_sort(test_arr)}\n")
```

```
#
==========================================================================
=
# TASK 2: BINARY SEARCH IMPLEMENTATION
#
==========================================================================
=


def binary_search(arr, target):
    """
    Searches for a target element in a sorted list using Binary Search.

    Best Case Complexity: O(1) - Element found at mid position on first check
Average Case Complexity: O(log n) - Element found after several divisions
    Worst Case Complexity: O(log n) - Element not found or at boundary
Space Complexity: O(1) - Only uses constant extra space

    Args:
        arr (list): Sorted list of comparable elements        target:
Element to search for

    Returns:        int: Index of target if found, -
1 otherwise
    """     left, right = 0, len(arr)
- 1


    while left <= right:
        mid = (left + right) // 2
```

```
        if arr[mid] == target:
            return mid        elif
arr[mid] < target:
            left = mid + 1
else:             right =
mid - 1

    return -1

# Test cases for Binary Search print("=" * 70)
print("TASK 2: BINARY SEARCH TESTING") print("=" *
70) sorted_array = [2, 5, 8, 12, 16, 23, 38, 45, 56,
67, 78] search_targets = [12, 2, 78, 100, 45]

print(f"Sorted Array: {sorted_array}\n") for
target in search_targets:
    index = binary_search(sorted_array, target)     if
index != -1:
        print(f"Target {target}: Found at index {index}")
else:         print(f"Target {target}: Not found (index -
1)")                    print()                                    #
==========================================
=================================
=
# TASK 3: HEALTHCARE APPOINTMENT SCHEDULING SYSTEM
#
```

```
==========================================================================
=


class Appointment:     def __init__(self, apt_id, patient_name, doctor_name, apt_time,
consultation_fee):
        self.apt_id = apt_id         self.patient_name
= patient_name         self.doctor_name =
doctor_name         self.apt_time = apt_time
self.consultation_fee = consultation_fee


    def __repr__(self):
        return (f"Apt({self.apt_id}, {self.patient_name}, {self.doctor_name}, "
f"{self.apt_time}, ${self.consultation_fee})")


def search_appointment_by_id(appointments, apt_id):
    """
    Binary Search for appointment by ID (requires sorted list).
    Time: O(log n), Space: O(1)
    """     left, right = 0, len(appointments)
- 1     while left <= right:
        mid = (left + right) // 2         if
appointments[mid].apt_id == apt_id:
return appointments[mid]     elif appointments[mid].apt_id
< apt_id:
            left = mid + 1
    else:
            right = mid - 1
return None
```

```
def sort_appointments_by_time(appointments):
    """
    Merge Sort by appointment time. Time: O(n log n), Space: O(n)
    """
    if len(appointments) <= 1:        return appointments
mid = len(appointments) // 2     left =
sort_appointments_by_time(appointments[:mid])     right =
sort_appointments_by_time(appointments[mid:])     return
merge_appointments_by_time(left, right)

def merge_appointments_by_time(left, right):
    result = []     i = j = 0     while i < len(left)
and j < len(right):        if left[i].apt_time
<= right[j].apt_time:

result.append(left[i])            i
+= 1        else:
result.append(right[j])
            j += 1
result.extend(left[i:])
result.extend(right[j:])     return
result

def sort_appointments_by_fee(appointments):
    """
    Merge Sort by consultation fee. Time: O(n log n), Space: O(n)
```

```
    """
    if len(appointments) <= 1:        return appointments
mid = len(appointments) // 2     left =
sort_appointments_by_fee(appointments[:mid])     right =
sort_appointments_by_fee(appointments[mid:])     return
merge_appointments_by_fee(left, right)

def merge_appointments_by_fee(left, right):
    result = []     i = j = 0     while i < len(left) and j < len(right):
if left[i].consultation_fee <= right[j].consultation_fee:

result.append(left[i])            i
+= 1        else:
result.append(right[j])
            j += 1
   result.extend(left[i:])
    result.extend(right[j:]) return
    result

# Test Healthcare System print("=" * 70) print("TASK 3: HEALTHCARE
APPOINTMENT SCHEDULING SYSTEM")
print("=" * 70) appointments
= [
    Appointment(103, "John Doe", "Dr. Smith", "2024-01-15 10:30", 150),
    Appointment(101, "Jane Smith", "Dr. Johnson", "2024-01-15 09:00", 120),
    Appointment(105, "Alice Brown", "Dr. Williams", "2024-01-15 14:00", 180),
    Appointment(102, "Bob Wilson", "Dr. Brown", "2024-01-15 11:00", 140)
]
```

```
appointments_sorted_id = sorted(appointments, key=lambda x: x.apt_id) search_result
= search_appointment_by_id(appointments_sorted_id, 103) print(f"Search Apt ID 103:
{search_result}")

print("\nSorted by Time:") for apt in sort_appointments_by_time(appointments):
    print(apt)

print("\nSorted by Fee:") for apt in sort_appointments_by_fee(appointments):
    print(apt) print()

#
==============================================================================
=
# TASK 4: RAILWAY TICKET RESERVATION SYSTEM
#
============================================================================== =

class Ticket:     def __init__(self, ticket_id, passenger_name, train_number, seat_number,
travel_date):
        self.ticket_id = ticket_id
self.passenger_name = passenger_name
self.train_number = train_number
self.seat_number = seat_number        self.travel_date
= travel_date

    def __repr__(self):
        return (f"Ticket({self.ticket_id}, {self.passenger_name}, Train#{self.train_number}, "
f"Seat{self.seat_number}, {self.travel_date})")
```

```
def search_ticket_by_id(tickets, ticket_id):
    """
    Binary Search for ticket by ID. Time: O(log n), Space: O(1)
    Recommended because ticket IDs are typically unique and searchable.
    """     left, right = 0, len(tickets)
- 1     while left <= right:
        mid = (left + right) // 2         if tickets[mid].ticket_id
== ticket_id:
return tickets[mid]     elif tickets[mid].ticket_id
< ticket_id:
            left = mid + 1         else:

            right = mid - 1
return None


def sort_tickets_by_date(tickets):
    """
    Merge Sort by travel date. Time: O(n log n), Space: O(n)
    Recommended for temporal sorting.
    """     if len(tickets) <= 1:         return
tickets     mid = len(tickets) // 2     left =
sort_tickets_by_date(tickets[:mid])     right =
sort_tickets_by_date(tickets[mid:])     return
merge_tickets_by_date(left, right)


def merge_tickets_by_date(left, right):
```

```
    result = []     i = j = 0     while i < len(left) and j
< len(right):        if left[i].travel_date <=
right[j].travel_date:

result.append(left[i])            i
+= 1        else:
result.append(right[j])

            j += 1
result.extend(left[i:])
result.extend(right[j:])     return
result

def sort_tickets_by_seat(tickets):
    """
    Merge Sort by seat number. Time: O(n log n), Space: O(n)
    """     if len(tickets) <= 1:        return
tickets     mid = len(tickets) // 2     left =
sort_tickets_by_seat(tickets[:mid])     right =
sort_tickets_by_seat(tickets[mid:])     return
merge_tickets_by_seat(left, right)

def merge_tickets_by_seat(left, right):
    result = []     i = j = 0     while i < len(left) and j < len(right):
if left[i].seat_number <= right[j].seat_number:
```

```
result.append(left[i])            i
+= 1        else:
result.append(right[j])
          j += 1
  result.extend(left[i:])
result.extend(right[j:])    return result


# Test Railway System print("=" * 70) print("TASK 4:
RAILWAY TICKET RESERVATION SYSTEM") print("=" *
70) tickets = [
   Ticket(104, "Charlie Davis", 201, 15, "2024-02-10"),
   Ticket(101, "Emma Wilson", 202, 5, "2024-02-08"),
   Ticket(103, "Frank Miller", 201, 12, "2024-02-10"),  Ticket(102, "Grace Lee", 203, 8, "2024-02-
09")
]


tickets_sorted_id = sorted(tickets, key=lambda x: x.ticket_id)
search_result = search_ticket_by_id(tickets_sorted_id, 103) print(f"Search
Ticket ID 103: {search_result}")


print("\nSorted by Travel Date:") for ticket
in sort_tickets_by_date(tickets):
   print(ticket)


print("\nSorted by Seat Number:") for ticket
in sort_tickets_by_seat(tickets):
   print(ticket) print()
```

#
==========
=

TASK 5: SMART HOSTEL ROOM ALLOCATION SYSTEM

#
==========
=

class RoomAllocation: def __init__(self, student_id, room_number, floor,

allocation_date):

  self.student_id = student_id

self.room_number = room_number

  self.floor = floor

self.allocation_date = allocation_date

  def __repr__(self):

  return (f"Allocation(StudentID:{self.student_id}, Room:{self.room_number}, "

f"Floor:{self.floor}, Date:{self.allocation_date})")

def search_allocation_by_id(allocations, student_id):

  """

  Binary Search for allocation by student ID. Time: O(log n), Space: O(1)

  Recommended for fast lookups on unique IDs.

  """ left, right = 0, len(allocations)

- 1 while left <= right:

  mid = (left + right) // 2 if allocations[mid].student_id

== student_id:

return allocations[mid] elif allocations[mid].student_id

< student_id:

```
            left = mid + 1        else:
            right = mid - 1
return None


def sort_allocations_by_room(allocations):
    """
    Merge Sort by room number. Time: O(n log n), Space: O(n)
    """     if len(allocations) <= 1:        return allocations
mid = len(allocations) // 2     left =
sort_allocations_by_room(allocations[:mid])     right =
sort_allocations_by_room(allocations[mid:])     return
merge_allocations_by_room(left, right)


def merge_allocations_by_room(left, right):
    result = []     i = j = 0     while i < len(left) and j < len(right):
if left[i].room_number <= right[j].room_number:


result.append(left[i])            i
+= 1        else:
            result.append(right[j])
            j += 1
result.extend(left[i:])
result.extend(right[j:])     return
result


def sort_allocations_by_date(allocations):
    """
```

```
    Merge Sort by allocation date. Time: O(n log n), Space: O(n)
    """     if len(allocations) <= 1:        return
allocations     mid = len(allocations) // 2     left =
sort_allocations_by_date(allocations[:mid])     right =
sort_allocations_by_date(allocations[mid:])     return
merge_allocations_by_date(left, right)

def merge_allocations_by_date(left, right):
    result = []     i = j = 0     while i < len(left) and j < len(right):
if left[i].allocation_date <= right[j].allocation_date:


result.append(left[i])            i
+= 1        else:
result.append(right[j])
            j += 1
   result.extend(left[i:])
    result.extend(right[j:]) return
    result


# Test Hostel System print("=" * 70) print("TASK 5: SMART HOSTEL
ROOM ALLOCATION SYSTEM")
print("=" * 70) allocations
= [
    RoomAllocation(105, 302, 3, "2024-01-20"),
    RoomAllocation(102, 201, 2, "2024-01-15"),
    RoomAllocation(104, 105, 1, "2024-01-18"),
    RoomAllocation(101, 102, 1, "2024-01-10")
]
```

```
allocations_sorted_id = sorted(allocations, key=lambda x: x.student_id)
search_result = search_allocation_by_id(allocations_sorted_id, 104) print(f"Search
Student ID 104: {search_result}")

print("\nSorted by Room Number:") for alloc in sort_allocations_by_room(allocations):
    print(alloc)


print("\nSorted by Allocation Date:") for alloc in sort_allocations_by_date(allocations):
    print(alloc) print()

#
==========================================================================
=
# TASK 6: ONLINE MOVIE STREAMING PLATFORM
#
==========================================================================
=


class Movie:     def __init__(self, movie_id, title, genre, rating, release_year):
        self.movie_id = movie_id
self.title = title        self.genre = genre
self.rating = rating
self.release_year = release_year


    def __repr__(self):
        return (f"Movie({self.movie_id}, {self.title}, {self.genre}, "                f"Rating:{self.rating},
{self.release_year})")

def search_movie_by_id(movies, movie_id):
```

```
    """
    Binary Search for movie by ID. Time: O(log n), Space: O(1)
    Recommended for unique movie IDs with fast lookups.
    """     left, right = 0, len(movies)
- 1     while left <= right:
        mid = (left + right) // 2        if
movies[mid].movie_id == movie_id:
return movies[mid]     elif movies[mid].movie_id
< movie_id:
            left = mid + 1        else:
            right = mid - 1
return None


def sort_movies_by_rating(movies):
    """
    Merge Sort by rating (descending). Time: O(n log n), Space: O(n)
    """     if len(movies) <= 1:        return movies
mid = len(movies) // 2     left =
sort_movies_by_rating(movies[:mid])     right =
sort_movies_by_rating(movies[mid:])     return
merge_movies_by_rating(left, right)


def merge_movies_by_rating(left, right):
    result = []     i = j = 0     while i < len(left)
and j < len(right):        if left[i].rating >=
right[j].rating:
```

```
result.append(left[i])            i
+= 1        else:
            result.append(right[j])
          j += 1
result.extend(left[i:])
result.extend(right[j:])     return
result


def sort_movies_by_year(movies):
    """
    Merge Sort by release year (descending). Time: O(n log n), Space: O(n)
    """     if len(movies) <= 1:        return
movies     mid = len(movies) // 2     left =
sort_movies_by_year(movies[:mid])     right =
sort_movies_by_year(movies[mid:])     return
merge_movies_by_year(left, right)


def merge_movies_by_year(left, right):
    result = []     i = j = 0     while i < len(left) and j < len(right):
if left[i].release_year >= right[j].release_year:


result.append(left[i])            i
+= 1        else:
result.append(right[j])
            j += 1
result.extend(left[i:])
result.extend(right[j:]) return result
```

```
# Test Movie Streaming System print("=" * 70)
print("TASK 6: ONLINE MOVIE STREAMING PLATFORM")
print("=" * 70) movies
= [
    Movie(105, "Inception", "Sci-Fi", 8.8, 2010),
    Movie(102, "The Matrix", "Sci-Fi", 8.7, 1999),
Movie(104, "Interstellar", "Sci-Fi", 8.6, 2014),
    Movie(101, "Avatar", "Sci-Fi", 7.8, 2009),
    Movie(103, "Pulp Fiction", "Drama", 8.9, 1994)
]

movies_sorted_id = sorted(movies, key=lambda x: x.movie_id)
search_result = search_movie_by_id(movies_sorted_id, 104) print(f"Search
Movie ID 104: {search_result}")

print("\nSorted by Rating (Highest First):") for
movie in sort_movies_by_rating(movies):
print(movie)


print("\nSorted by Release Year (Newest First):") for
movie in sort_movies_by_year(movies):
    print(movie) print()

#
==============================================================================
=
# TASK 7: SMART AGRICULTURE CROP MONITORING SYSTEM
#
```

```
==========================================================================
=


class Crop:     def __init__(self, crop_id, crop_name, soil_moisture, temperature, yield_estimate):
        self.crop_id = crop_id         self.crop_name
= crop_name         self.soil_moisture =
soil_moisture         self.temperature =
temperature         self.yield_estimate =
yield_estimate


    def __repr__(self):
        return (f"Crop({self.crop_id}, {self.crop_name}, Moisture:{self.soil_moisture}%, "
f"Temp:{self.temperature}°C, Yield:{self.yield_estimate}kg)")


def search_crop_by_id(crops, crop_id):
    """
    Binary Search for crop by ID. Time: O(log n), Space: O(1)
    Recommended for rapid crop lookups.
    """     left, right = 0, len(crops)
- 1     while left <= right:
        mid = (left + right) // 2      if
    crops[mid].crop_id == crop_id:
            return crops[mid]     elif crops[mid].crop_id
    < crop_id:
            left = mid + 1         else:
            right = mid - 1
return None
```

```
def sort_crops_by_moisture(crops):
    """
    Merge Sort by soil moisture level. Time: O(n log n), Space: O(n)
    """     if len(crops) <= 1:         return crops
mid = len(crops) // 2     left =
sort_crops_by_moisture(crops[:mid])     right =
sort_crops_by_moisture(crops[mid:])     return
merge_crops_by_moisture(left, right)

def merge_crops_by_moisture(left, right):
    result = []     i = j = 0     while i < len(left) and j < len(right):
if left[i].soil_moisture <= right[j].soil_moisture:


result.append(left[i])            i
+= 1     else:
result.append(right[j])

            j += 1
result.extend(left[i:])
result.extend(right[j:])     return
result

def sort_crops_by_yield(crops):
    """
    Merge Sort by yield estimate (descending). Time: O(n log n), Space: O(n)
```

```
    """     if len(crops) <= 1:         return
crops     mid = len(crops) // 2     left =
sort_crops_by_yield(crops[:mid])     right =
sort_crops_by_yield(crops[mid:])     return
merge_crops_by_yield(left, right)

def merge_crops_by_yield(left, right):
    result = []     i = j = 0     while i < len(left) and j < len(right):
if left[i].yield_estimate >= right[j].yield_estimate:

result.append(left[i])            i
+= 1         else:
            result.append(right[j])         j
    += 1 result.extend(left[i:])
    result.extend(right[j:]) return
    result

# Test Agriculture System print("=" * 70) print("TASK 7: SMART AGRICULTURE
CROP MONITORING SYSTEM")
print("=" * 70) crops
= [
    Crop(105, "Wheat", 45, 22, 5000),
    Crop(102, "Rice", 60, 25, 6000),
    Crop(104, "Corn", 50, 24, 7000),
    Crop(101, "Barley", 35, 18, 4000),
    Crop(103, "Soybean", 55, 23, 3500)
]
```

```
crops_sorted_id = sorted(crops, key=lambda x: x.crop_id) search_result
= search_crop_by_id(crops_sorted_id, 103) print(f"Search Crop ID 103:
{search_result}")

print("\nSorted by Soil Moisture (Low to High):") for
crop in sort_crops_by_moisture(crops):
    print(crop)

print("\nSorted by Yield Estimate (Highest First):") for
crop in sort_crops_by_yield(crops):
    print(crop)  print()

#
==========================================================================
=
# TASK 8: AIRPORT FLIGHT MANAGEMENT SYSTEM
#
==========================================================================
=

class Flight:     def __init__(self, flight_id, airline_name, departure_time, arrival_time,
status):
        self.flight_id = flight_id
self.airline_name = airline_name
self.departure_time = departure_time
self.arrival_time = arrival_time         self.status
= status
```

```
    def __repr__(self):
        return (f"Flight({self.flight_id}, {self.airline_name}, Depart:{self.departure_time}, "
f"Arrive:{self.arrival_time}, {self.status})")


def search_flight_by_id(flights, flight_id):
    """
    Binary Search for flight by ID. Time: O(log n), Space: O(1)
    Recommended for quick flight lookups at airport counters.
    """     left, right = 0, len(flights)
- 1     while left <= right:

        mid = (left + right) // 2 if flights[mid].flight_id
        == flight_id:
            return flights[mid]     elif flights[mid].flight_id
    < flight_id:
            left = mid + 1        else:
            right = mid - 1
return None


def sort_flights_by_departure(flights):
    """
    Merge Sort by departure time. Time: O(n log n), Space: O(n)
    Recommended for scheduling and display purposes.
    """     if len(flights) <= 1:        return flights
mid = len(flights) // 2     left =
sort_flights_by_departure(flights[:mid])     right =
sort_flights_by_departure(flights[mid:])     return
merge_flights_by_departure(left, right)
```

```
def merge_flights_by_departure(left, right):
    result = []     i = j = 0     while i < len(left) and j < len(right):
if left[i].departure_time <= right[j].departure_time:
            result.append(left[i])  i
            += 1 else:
  result.append(right[j])

      j      += 1
  result.extend(left[i:])
  result.extend(right[j:])     return
  result


def sort_flights_by_arrival(flights):
    """
    Merge Sort by arrival time. Time: O(n log n), Space: O(n)
    """     if len(flights) <= 1:        return flights
mid = len(flights) // 2     left =
sort_flights_by_arrival(flights[:mid])     right =
sort_flights_by_arrival(flights[mid:])     return
merge_flights_by_arrival(left, right)


def merge_flights_by_arrival(left, right):
    result = []     i = j = 0     while i < len(left) and j <
len(right):        if left[i].arrival_time <=
right[j].arrival_time:
```

```
result.append(left[i])            i
+= 1        else:
result.append(right[j])     j
+= 1 result.extend(left[i:])
result.extend(right[j:])
return result

# Test Airport Flight System print("=" * 70) print("TASK 8: AIRPORT
FLIGHT MANAGEMENT SYSTEM")
print("=" * 70) flights
= [
    Flight(105, "Delta", "14:30", "18:45", "On-Time"),
    Flight(102, "United", "09:15", "13:30", "Delayed"),
    Flight(104, "American", "12:00", "16:20", "On-Time"),
    Flight(101, "Southwest", "08:00", "12:15", "On-Time"),
    Flight(103, "JetBlue", "11:45", "15:00", "Cancelled")
]

flights_sorted_id = sorted(flights, key=lambda x: x.flight_id) search_result
= search_flight_by_id(flights_sorted_id, 104) print(f"Search Flight ID 104:
{search_result}")

print("\nSorted by Departure Time:") for flight
in sort_flights_by_departure(flights):
    print(flight)

print("\nSorted by Arrival Time:")  for
flight in sort_flights_by_arrival(flights):
```

```
print(flight)


print("=" * 70) print("ALL TASKS COMPLETED

SUCCESSFULLY") print("=" * 70)
```

OUTPUT:

```
PROBLEMS 533   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   QUERY RESULTS   Python

sanjaykumar@perumandlas-MacBook-Air AI - assiant %  source "/Users/sanjaykumar/Documents/AI - assiant/.venv/bin/acti
vate"
"/Users/sanjaykumar/Documents/AI - assiant/.venv/bin/python" "/Users/sanjaykumar/Documents/AI - assiant/ass-12.5.py"
(.venv) sanjaykumar@perumandlas-MacBook-Air AI - assiant % "/Users/sanjaykumar/Documents/AI - assiant/.venv/bin/pyth
on" "/Users/sanjaykumar/Documents/AI - assiant/ass-12.5.py"
======================================================================
TASK 1: MERGE SORT TESTING
======================================================================
Original: [64, 34, 25, 12, 22, 11, 90]
Sorted:   [11, 12, 22, 25, 34, 64, 90]

Original: [5, 2, 8, 1, 9]
Sorted:   [1, 2, 5, 8, 9]

Original: [1]
Sorted:   [1]

Original: []
Sorted:   []

Original: [10, 10, 10, 10]
Sorted:   [10, 10, 10, 10]

======================================================================
TASK 2: BINARY SEARCH TESTING
======================================================================
Sorted Array: [2, 5, 8, 12, 16, 23, 38, 45, 56, 67, 78]

Target 12: Found at index 3
Target 2: Found at index 0
Target 78: Found at index 10
Target 100: Not found (index -1)
Target 45: Found at index 7
```

```
======================================================================
TASK 3: HEALTHCARE APPOINTMENT SCHEDULING SYSTEM
======================================================================
Search Apt ID 103: Apt(103, John Doe, Dr. Smith, 2024-01-15 10:30, $150)

Sorted by Time:
Apt(101, Jane Smith, Dr. Johnson, 2024-01-15 09:00, $120)
Apt(103, John Doe, Dr. Smith, 2024-01-15 10:30, $150)
Apt(102, Bob Wilson, Dr. Brown, 2024-01-15 11:00, $140)
Apt(105, Alice Brown, Dr. Williams, 2024-01-15 14:00, $180)

Sorted by Fee:
Apt(101, Jane Smith, Dr. Johnson, 2024-01-15 09:00, $120)
Apt(102, Bob Wilson, Dr. Brown, 2024-01-15 11:00, $140)
Apt(103, John Doe, Dr. Smith, 2024-01-15 10:30, $150)
Apt(105, Alice Brown, Dr. Williams, 2024-01-15 14:00, $180)

======================================================================
TASK 4: RAILWAY TICKET RESERVATION SYSTEM
======================================================================
Search Ticket ID 103: Ticket(103, Frank Miller, Train#201, Seat12, 2024-02-10)

Sorted by Travel Date:
Ticket(101, Emma Wilson, Train#202, Seat5, 2024-02-08)
Ticket(102, Grace Lee, Train#203, Seat8, 2024-02-09)
Ticket(104, Charlie Davis, Train#201, Seat15, 2024-02-10)
Ticket(103, Frank Miller, Train#201, Seat12, 2024-02-10)

Sorted by Seat Number:
Ticket(101, Emma Wilson, Train#202, Seat5, 2024-02-08)
Ticket(102, Grace Lee, Train#203, Seat8, 2024-02-09)
Ticket(103, Frank Miller, Train#201, Seat12, 2024-02-10)
Ticket(104, Charlie Davis, Train#201, Seat15, 2024-02-10)

----------------------------------------------------------------------
```

```
(.venv) sanjaykumar@perumandlas-MacBook-Air AI - assiant % "/Users/sanjaykumar/Documents/AI - assiant/.venv/bin/pyth
on" "/Users/sanjaykumar/Documents/AI - assiant/ass-12.5.py"
TASK 5: SMART HOSTEL ROOM ALLOCATION SYSTEM
======================================================================
Search Student ID 104: Allocation(StudentID:104, Room:105, Floor:1, Date:2024-01-18)

Sorted by Room Number:
Allocation(StudentID:101, Room:102, Floor:1, Date:2024-01-10)
Allocation(StudentID:104, Room:105, Floor:1, Date:2024-01-18)
Allocation(StudentID:102, Room:201, Floor:2, Date:2024-01-15)
Allocation(StudentID:105, Room:302, Floor:3, Date:2024-01-20)

Sorted by Allocation Date:
Allocation(StudentID:101, Room:102, Floor:1, Date:2024-01-10)
Allocation(StudentID:102, Room:201, Floor:2, Date:2024-01-15)
Allocation(StudentID:104, Room:105, Floor:1, Date:2024-01-18)
Allocation(StudentID:105, Room:302, Floor:3, Date:2024-01-20)

======================================================================
TASK 6: ONLINE MOVIE STREAMING PLATFORM
======================================================================
Search Movie ID 104: Movie(104, Interstellar, Sci-Fi, Rating:8.6, 2014)

Sorted by Rating (Highest First):
Movie(103, Pulp Fiction, Drama, Rating:8.9, 1994)
Movie(105, Inception, Sci-Fi, Rating:8.8, 2010)
Movie(102, The Matrix, Sci-Fi, Rating:8.7, 1999)
Movie(104, Interstellar, Sci-Fi, Rating:8.6, 2014)
Movie(101, Avatar, Sci-Fi, Rating:7.8, 2009)

Sorted by Release Year (Newest First):
Movie(104, Interstellar, Sci-Fi, Rating:8.6, 2014)
Movie(105, Inception, Sci-Fi, Rating:8.8, 2010)
Movie(101, Avatar, Sci-Fi, Rating:7.8, 2009)
Movie(102, The Matrix, Sci-Fi, Rating:8.7, 1999)
Movie(103, Pulp Fiction, Drama, Rating:8.9, 1994)

======================================================================
```

```
==========================================================================
TASK 7: SMART AGRICULTURE CROP MONITORING SYSTEM
==========================================================================
Search Crop ID 103: Crop(103, Soybean, Moisture:55%, Temp:23°C, Yield:3500kg)

Sorted by Soil Moisture (Low to High):
Crop(101, Barley, Moisture:35%, Temp:18°C, Yield:4000kg)
Crop(105, Wheat, Moisture:45%, Temp:22°C, Yield:5000kg)
Crop(104, Corn, Moisture:50%, Temp:24°C, Yield:7000kg)
Crop(103, Soybean, Moisture:55%, Temp:23°C, Yield:3500kg)
Crop(102, Rice, Moisture:60%, Temp:25°C, Yield:6000kg)

Sorted by Yield Estimate (Highest First):
Crop(104, Corn, Moisture:50%, Temp:24°C, Yield:7000kg)
Crop(102, Rice, Moisture:60%, Temp:25°C, Yield:6000kg)
Crop(105, Wheat, Moisture:45%, Temp:22°C, Yield:5000kg)
Crop(101, Barley, Moisture:35%, Temp:18°C, Yield:4000kg)
Crop(103, Soybean, Moisture:55%, Temp:23°C, Yield:3500kg)
```

```
==========================================================================
TASK 8: AIRPORT FLIGHT MANAGEMENT SYSTEM
==========================================================================
Search Flight ID 104: Flight(104, American, Depart:12:00, Arrive:16:20, On-Time)

Sorted by Departure Time:
Flight(101, Southwest, Depart:08:00, Arrive:12:15, On-Time)
Flight(102, United, Depart:09:15, Arrive:13:30, Delayed)
Flight(103, JetBlue, Depart:11:45, Arrive:15:00, Cancelled)
Flight(104, American, Depart:12:00, Arrive:16:20, On-Time)
Flight(105, Delta, Depart:14:30, Arrive:18:45, On-Time)

Sorted by Arrival Time:
Flight(101, Southwest, Depart:08:00, Arrive:12:15, On-Time)
Flight(102, United, Depart:09:15, Arrive:13:30, Delayed)
Flight(103, JetBlue, Depart:11:45, Arrive:15:00, Cancelled)
Flight(104, American, Depart:12:00, Arrive:16:20, On-Time)
Flight(105, Delta, Depart:14:30, Arrive:18:45, On-Time)
==========================================================================
ALL TASKS COMPLETED SUCCESSFULLY
==========================================================================
(.venv) sanjaykumar@perumandlas-MacBook-Air AI - assiant %
```